

IntelliJ + Maven: PRIME v3 - Local Automation Setup



Overview

This guide provides step-by-step instructions for setting up the Prime v3 automation framework locally using IntelliJ IDEA. Prime v3 is Maven-based and uses Java 17+, making it easier to set up compared to the legacy Prime v2 framework.

*Scope: Prime **version 3** local setup for the **Automation** repo (separate from API and shared libraries). This guide covers prerequisites, cloning from GitLab via IntelliJ/CLI/SourceTree, IntelliJ import, Nexus, host properties, Maven builds /profiles, and FAQs.*

0) Repo layout (context)

- **Automation project** (this guide) - can be located at: <https://gitlab.com/ascensus-gs/products/depot/qa-automation/prime-test-automation>
- **Common libraries:** qa - resource (shared utilities/resources) and PRIME framework modules (prime-parent, prime-core, etc.) live in separate repos and are consumed via Maven/Nexus in v3.

*In v3, you usually **do not need** to build qa-resource or prime-parent locally; they're published to **Nexus** from CI. Local builds are still possible if you need to test framework changes.*

1) Prerequisites

Required Software

Tool	Version	Installation Notes
Java JDK	17+	Download from Oracle/OpenJDK Installed and set as JAVA_HOME; add %JAVA_HOME%\bin to PATH.
Maven	3.9.3+	Required for dependency management installed (mvn -v).
IntelliJ IDEA	Latest	Community (free) or Ultimate
Git	Latest	For repository cloning
Selenium	4.6+	auto-resolves WebDrivers; no manual driver config unless you prefer.

Verify

```
java -version
mvn -v
```

System Environment Setup

Java Configuration *(Steps applicable only if Java is missing)*

1. Download and install Java JDK 17+ from [Oracle](#) or [OpenJDK](#)
2. Set system environment variables:
 - JAVA_HOME = C:\Program Files\Java\jdk-17.0.2 (adjust path as needed)
 - Add %JAVA_HOME%\bin to system PATH
3. Verify installation: Open command prompt and run `java -version`

Maven Configuration *(Steps applicable only if Maven is missing)*

1. Download Maven from [Apache Maven](#)
2. Extract to a directory (e.g., C:\Program Files\Apache\maven)
3. Set system environment variables:
 - MAVEN_HOME = Maven installation directory
 - Add %MAVEN_HOME%\bin to system PATH
4. Verify installation: Run `mvn -version` in command prompt

Nexus Repository Setup

if getting following issue on maven build & unable to find Nexus artifact then follow these steps

```
[INFO] prime-parent ..... SUCCESS [ 1.188 s]
[INFO] prime-core ..... FAILURE [ 1.192 s]
[INFO] prime-cucumber ..... SKIPPED
[INFO] prime-lib ..... SKIPPED
[INFO] -----
[INFO] BUILD FAILURE
[INFO] -----
[INFO] Total time: 2.590 s
[INFO] Finished at: 2025-08-12T11:08:47-04:00
[INFO] -----
[ERROR] Failed to execute goal on project prime-core: Could not resolve dependencies for project prime:prime-core:jar:3.0.0: The following
artifacts could not be resolved: qa-resource:qa-resource-database-loader:jar:1.0.1 (absent), qa-resource:qa-resource-data-generation:jar:
(absent): qa-resource:qa-resource-database-loader:jar:1.0.1 was not found in https://repo.maven.apache.org/maven2 during a previous atten
This failure was cached in the local repository and resolution is not reattempted until the update interval of central has elapsed or upd
are forced -> [Help 1]
[ERROR]
```

1. Download settings.xml from: L:\Departments\Financial Services\UII\UII\UII-BODEV\unite_microservice_setup_files
2. Copy to: C:\Users\<username>\.m2\settings.xml
3. This lets Maven resolve PRIME/qa-resource artifacts from Nexus during builds.

If you skip this step, first project build will fail resolving parent POMs/dependencies.

Install IntelliJ IDEA (Steps applicable only if IntelliJ is missing)

1. Download IntelliJ IDEA from [JetBrains](#)
2. Choose Community (free) or Ultimate edition
3. Ensure 64-bit version matches your Java installation
4. Install with default settings

2) Clone the Automation repo

Pick **one** method.

A) IntelliJ – Get from Version Control

- Open IntelliJ IDEA
- Select **Get from Version Control**
- Enter repository URL: <https://gitlab.com/ascensus-gs/products/depot/qa-automation/prime-test-automation>
- Choose local directory for cloning
- Click **Clone**
- Authenticate with GitLab credentials when prompted

B) Git CLI/Using Command Line (Git Bash)

```
cd C:\workspace # or your preferred directory
git clone https://gitlab.com/ascensus-gs/products/depot/qa-automation/prime-test-automation.git
cd automation
```

C) SourceTree

- Open SourceTree
- Click **Clone**
- Enter repository URL: <https://gitlab.com/ascensus-gs/products/depot/qa-automation/prime-test-automation>
- Set destination path
- Click **Clone**

3) Open/import in IntelliJ (Maven project)

1. Open IntelliJ **File** **Open** select the repo folder **Open**.

2. Accept “**Autoimport Maven**” / “Download dependencies”.
3. Check **Maven** settings: *Settings Build, Execution, Deployment Build Tools Maven* use **Bundled** (or your external Maven) and make sure the JDK for Maven is **Java 17**.
4. Set **Project SDK**: *File Project Structure Project SDK: Java 17; Language level: 17*.
5. Plugins (recommended): **Cucumber for Java**; **TestNG** plugin or dependency for nicer results navigation.

4) Create your host properties file (personal overrides)

Why Host Properties?

The host properties file contains personal configurations that override environment defaults (database credentials, browser settings, etc.).

Setup Steps

1. Navigate to project's config folder: `src/test/resources/config/`
2. Create file named: `<COMPUTERNAME>.properties` or `<HOSTNAME>.properties` *e.g. LT12800.properties*
 - a. PRIME v3 loads two property files on start:
 - i. **Environment properties** (projectshared)
 - ii. **Host/personal properties** (your local overrides)
3. Add personal configurations:

properties

```
# Database Configuration
UNITEDDATABASEURL=localhost:41521:UIIS01
UNITEUSERNAME=<DB username>
UNITEPASSWORD=<your password>

# Browser Settings (required)
BROWSER=chrome

# Browser Settings (optional - override environment file)
WAITTIME=1000
IMPLICITWAITTIME=30
HEADLESS=FALSE
FEEDWAITTIME=60000
SLEEPTIME=1000
```

***Important:** Never commit host properties files to version control!*

***Note:** If you skip host properties, PRIME will fall back to the project's default config file; use a host file to keep creds and machine settings out of source control.*

***Important:** Keep personal files **untracked**. Your POM/profile likely already ignores these.*

5) First build paths (choose one)

Option A (recommended): Use Nexuspublished artifacts

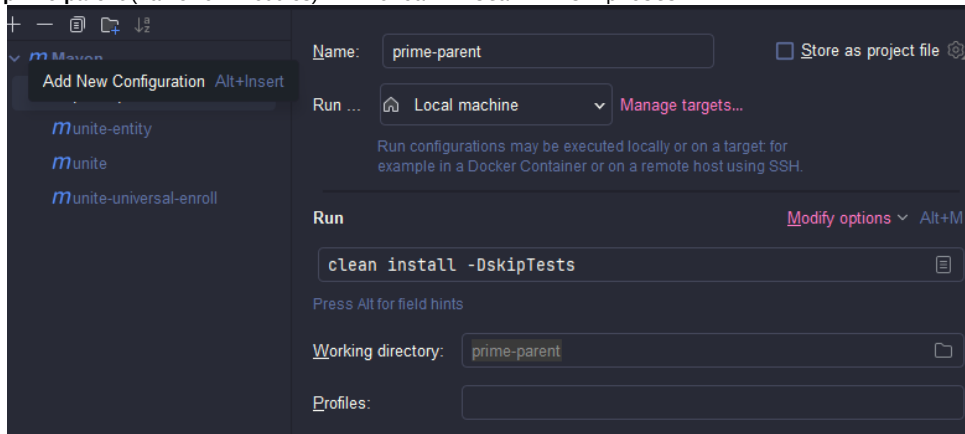
- Simply build the automation project; parent/framework libs will be pulled from Nexus.

Option B (optional): Local framework builds

Sometimes you'll need local edits to shared libs or you're offline. Build these in order:

1. **qa-resource**: `mvn clean install -DskipTests`

2. **prime-parent** (framework modules): `mvn clean install -DskipTests`



3. (Optional) In your automation project, copy framework resources/docs: `mvn clean generate-resources`

6) Build the project (Automation)

Terminal (from project root):

```
mvn clean install
```

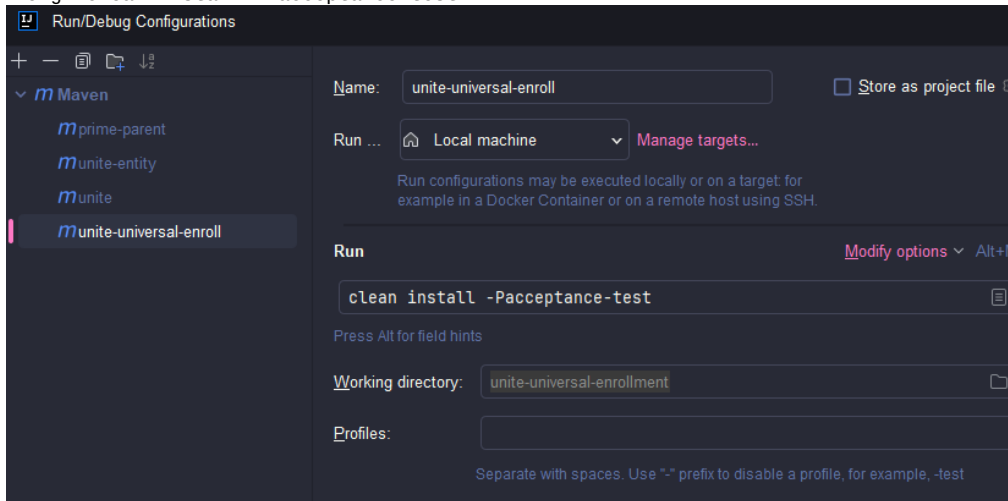
IntelliJ Run Configuration (Maven)

- **Working dir:** project root
- **Command:** `clean install`
- (Optional) **Profiles:** set the profile you want (see below)

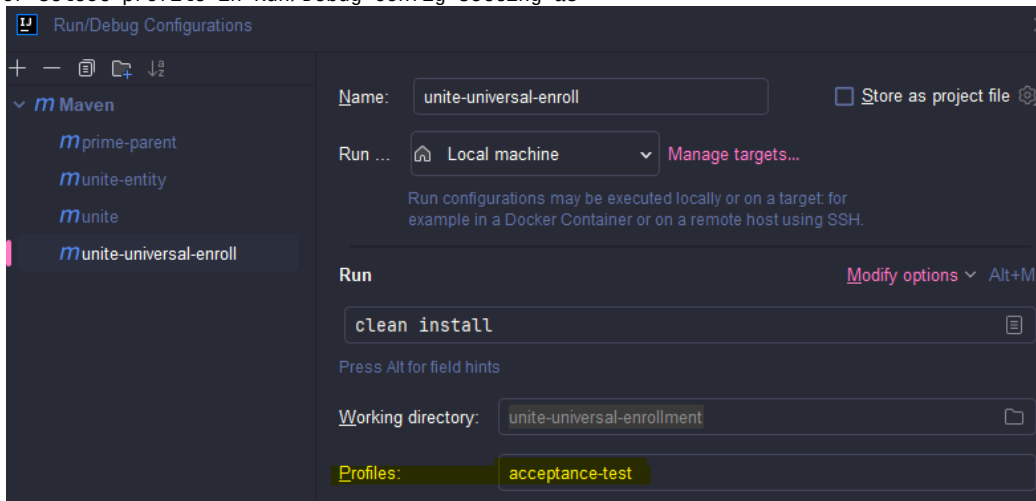
Profiles

- Projects define named profiles for different suites/environments. Examples: `acceptance-test`, `local-run`, `smoke`, `nightly` (actual names live in the project's `pom.xml`).
- Run with a profile:

```
mvn clean install -P<profile-id>
# e.g. clean install -Pacceptance-test
```



or select profile in Run/Debug Config setting as



Override suites/properties at runtime (advanced)

```
# override which TestNG suite(s) to run (comma separated paths under /bin)
mvn clean install -Dxml.suite=bin/frontoffice/smoke.xml
```

```
# override which config files to load
mvn clean install -DenvProperties=stage.properties -DhostProperties=YOURHOST.properties
```

Creating Run Configurations in IntelliJ

1. **Run** Edit Configurations
2. **Add New Maven**
3. Set working directory to project root
4. Command line: `clean install -P<profile-name>`
5. Save configuration for reuse

7) Running tests

- **Default:** `mvn clean install` runs the default test profile defined in the POM.
- Specific profile: `mvn clean install -P<profile-id>` (acceptance-test/smoke/etc.). e.g. `mvn clean install -Pacceptancetest`
- **From IntelliJ:** rightclick a **TestNG** suite XML under `/bin` and **Run**, or create a **Maven** run config as above.
- **Run a specific Scenario/Example:** add `<line>` to the feature path in the TestNG XML (see project runners). Example: `.../UniversalEnrollmentPositive.feature:26`

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
3  <suite verbose="1" name="UE Test Suite" parallel="tests" thread-count="1">
4      <listeners>
5          <listener class-name="core.listener.PrimeResourceManager"/>
6      </listeners>
7      <test name="Stage 1 Testing">
8          <parameter name="traunch" value="mdd"/>
9          <parameter name="features" value="frontoffice/enrollment/common/feature/UniversalEnrollmentPositive.feature:26" />
10         <classes>
11             <class name="core.runner.FeatureRunner"/>
12         </classes>
13     </test>
14 </suite>

```

8) IntelliJ tips

- After first import, run **Maven Reimport** if dependencies look red.
- If the IDE crawls, try **File Invalidate Caches / Restart**.
- Ensure **Git** is configured (Settings Version Control Git) and GitLab server is added under **Settings GitLab**.

9) Install Required Plugins

Cucumber for Java

1. **File Settings Plugins Marketplace**
2. Search: "Cucumber for Java"
3. Click **Install**
4. Restart IntelliJ

TestNG (if not included)

1. **File Settings Plugins Marketplace**
2. Search: "TestNG"
3. Click **Install**
4. Restart IntelliJ

10) Install Optional Plugins

Lombok

1. **File Settings Plugins Marketplace**
2. Search: "Lombok"
3. Click **Install**
4. Restart IntelliJ

11) Frequently Asked Questions (FAQ)

Q1) Maven can't resolve the parent POM (prime-parent) from Nexus.

A: Confirm your `~/m2/settings.xml` exists and has the team Nexus servers/credentials. If still failing, run with `-U` to force updates. Also ensure shared libs were published recently.

Q2) Cannot instantiate class `core.listener.ResourceManager` at startup.

A: Your configuration files failed to load or required properties are missing. Check your `envProperties` and `hostProperties` settings and file locations. Ensure your host file name matches your machine and is placed under `src/test/resources/config`.

Q3) Chrome/Driver mismatch errors (e.g., "This version of ChromeDriver only supports Chrome ...").

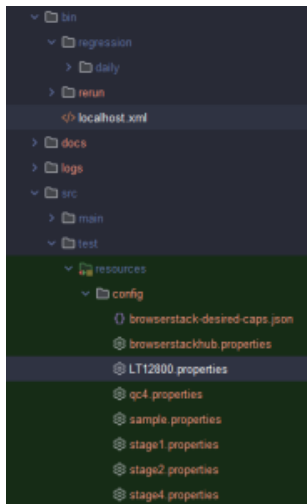
A: For v3, remove old `%SELENIUM_HOME%/PATH` driver entries to let Selenium **automanage** the right driver. Or, if you force a driver on `PATH`, make sure it matches your browser version and the Selenium `selenium-java` lib version in the framework.

Q4) Maven error: `modelVersion '3.6.3' is older/unsupported or your Maven is too old.`

A: Install/upgrade **Maven 3.9.3+** and try again.

Q5) Where do I put TestNG XMLs and configs?

A: Project convention: TestNG suites live under `/bin` (you can have multiple). Config files are under `src/test/resources/config`. Security files (e.g., PEM) go under `src/test/resources/security`. in case config folder is not visible then build a project, it will be created automatically



Q6) How do I run just one feature or a subset?

A: In the TestNG XML, set the features parameter to the feature path and suffix with :<line> for specific Scenario/Example lines. You can specify multiple line numbers separated by " : ".

Q7) Where do I find the test report/logs?

A: Maven Surefire creates a TestNG report per module under target/surefire-reports. Console/file logging levels are controlled by Logback properties (see project docs). You can also enable -Dlogging.console=DEBUG for more verbosity.

Q8) I don't see my personal overrides taking effect.

A: Verify which properties file names your POM/profile is passing as envProperties/hostProperties. Make sure your host file name matches your machine env var and that the profile you're running actually passes it. You can also override on the command line (see examples above).

Q9) IDE can't find step definitions from feature files.

A: Install **Cucumber for Java** plugin; ensure modules with step defs are part of the project; sync Maven; reindex (**Invalidate Caches**); ensure language level/JDK is set correctly.

Q10) Do I need to build qa-resource and prime-parent every time?

A: No. In v3, CI publishes these to Nexus and your project consumes them. Only build locally if you're developing the framework itself or troubleshooting Nexus.

Q11) Build fails with "Cannot instantiate class core.listener.ResourceManager"

A: This indicates configuration file issues.

- Verify host properties file exists and is properly named
- Check environment.properties is correctly specified in Maven profile
- Ensure all required properties are set

Q12) Tests run but no browser opens

A: WebDriver configuration issues.

- Check host properties for HEADLESS setting
- Verify browser is installed and accessible
- Review WebDriver logs for specific errors

Q13) GitLab authentication issues

A: Credential problems.

- Use Personal Access Token (PAT) instead of password
- Configure Git credentials: `git config --global user.name "Your Name"`
- Consider using SSH keys for authentication

Troubleshooting Common Issues

Issue	Symptoms	Solution
IDE Slowness	IntelliJ freezes or slow response	File Invalidate Caches Restart
Dependency Not Found	ClassNotFoundException	Reimport Maven project, verify build order
Git Authentication	Push/pull failures	Use PAT or SSH key authentication

Memory Issues	OutOfMemoryError	Increase IntelliJ heap size in Help Change Memory Settings
Plugin Conflicts	Feature files not recognized	Reinstall Cucumber plugin, ensure proper project structure

11) Quick reference

- **Firsttime**

1. Install Java 17, Maven 3.9.3, IntelliJ.
2. Put team settings.xml at ~/.m2/.
3. Clone Automation repo; open in IntelliJ; set SDK 17; install Cucumber/TestNG plugins.
4. Create your host file in src/test/resources/config/.
5. mvn clean install (or -P<profile-id>).

- **Useful commands**

```
# Build everything (default profile)
mvn clean install

# Acceptance/smoke/etc. (replace with your project's profile)
mvn clean install -Paccepttest

# Force update dependencies from Nexus
mvn clean install -U

# Skip unit tests when building shared libs locally
mvn clean install -DskipTests

# Override suites and property files
mvn clean install -Dxml.suite=bin/smoke.xml -DenvProperties=stage.properties -DhostProperties=MYPC.properties
```

12) SDET Syllabus (first 1–2 weeks)

Day 1–2

- Environment & tool setup (this guide)
- Read **Prime v3 General Documentation** sections on **Configuration Files**, **Suite Runner**, **Resource Manager**, **Test Case Loading**, **Element Locators**.

Day 3–4

- Run an existing suite locally; explore bin/*.xml and src/test/resources/testsuite/** structure
- Read **Folder Structure & Naming**

Day 5–7

- Create/modify a simple feature + CSV; run via a dedicated profile
- Review **Automation Best Practices** (externalize SQL/errors; scalable locators; reusable code; data generation)

Stretch

- Debugging: enable -Dmaven.surefire.debug and attach IntelliJ remote debugger (port 5005)
- Try BrowserStack/Grid if your team uses them